An assistant that handles long conversations and any file in, any file out

Design and ship a working AI chat product end to end. We care less about feature count and more about how you make it reliable, how you reason about trade-offs, and whether a real user would enjoy using it.

STACK	TIMEBOX	LLM	SUBMIT
LangGraph · Django · React	**2 days**	**Your choice**	**Repo + demo video**
Mandatory	Aim for ~12–16 focused hours	Any provider; key can be supplied	Followed by a 60-min review

01 Core requirements

AREA	WHAT IT MUST DO
Long-running chat	Multi-turn threads that stay coherent across hundreds of messages and beyond the model's context window (summarisation, trimming or retrieval: your call, justify it). Threads persist and resume after refresh or restart. Responses stream token by token; user can stop and regenerate.
Any file in	Upload one or many files in a message: PDF, DOCX, XLSX/CSV, PPTX, images, plain text/code, JSON, ZIP at minimum. The assistant must actually use their contents (e.g. summarise a PDF, analyse a sheet, describe an image) and refer back to them later in the thread. Unsupported types fail gracefully.
Any file out	The assistant creates real files on request: DOCX, PDF, XLSX, CSV, PPTX, PNG chart, Markdown and code files at minimum. Files appear as downloadable cards in the chat with a preview where sensible, and can be revised ("add a column", "make it shorter").
Agent (LangGraph)	A graph-based agent with explicit nodes/tools for file parsing, file generation and answering. Use a checkpointer for state. Tool failures and bad model output are retried or surfaced, never silently dropped.
Backend (Django)	Auth (simple is fine), thread/message/file models, file storage, streaming endpoint (SSE or WebSockets), size and type limits, basic rate limiting.
Frontend (React)	Thread list, chat view with Markdown/code rendering, drag-and-drop upload with progress, file chips, loading/error/empty states. Usable on mobile width.

02 Deliverables

- ☐ Git repo with `README.md`: setup in ≤5 commands (Docker Compose preferred), env vars, architecture diagram
- ☐ A short `DECISIONS.md`: memory strategy, file pipeline, what you cut and why, what you'd do next
- ☐ Tests for the critical paths (file parsing, generation, graph routing)
- ☐ 5–8 min screen recording covering a long thread, a multi-file upload and a generated file being revised
- ☐ Rough cost per conversation and where it would break at 1,000 concurrent users

03 Stretch (optional)

- Background jobs for heavy files (Celery/RQ) with progress in the UI
- Human-in-the-loop: agent asks for confirmation before large generations
- Sandboxed code execution for data analysis
- Tracing/observability (LangSmith or equivalent) and token usage per thread

04 How we evaluate

CRITERION	WEIGHT
Works end to end: requirements in 01 behave as described	**25%**
Architecture: graph design, memory strategy, clean Django/React boundaries	**20%**
Product & UX: streaming feel, file handling, states and edge cases	**20%**
Reliability: errors, retries, limits, security of uploads	**15%**
Code quality & tests	**10%**
Communication: README, decisions, review discussion	**10%**

> **Ground rules.** AI coding tools are allowed; you must be able to explain every line. Prefer a smaller scope done well over everything half-done, and tell us what you left out. Questions are welcome at any point; asking good ones counts in your favour.